

Mexi Banco con Docker Compose

Pablo Vaquero · Grupo 2 · FI, UNAM · Semestre 2027-1

Sesión: sábado 19 de septiembre · Ejecución: 26 de septiembre de 2026

1. Clonar la versión del laboratorio

Se clonó el repositorio del profesor con la etiqueta **v05.1**. Se verificó el commit 88203161aa31; la copia queda en *detached HEAD* porque apunta a una etiqueta. No se modificó el código de la aplicación.

```
git clone --branch v05.1 --depth 1 https://github.com/OscarRuiz21/mexi-banco.git mexi-banco-s05
cd mexi-banco-s05
git describe --tags --exact-match
```

2. Construir y levantar los servicios

Se ejecutó `docker compose up --build -d --wait` y luego `docker compose ps`. Tanto **app** como **db** alcanzaron el estado **healthy**. La opción `--wait` espera esa condición antes de devolver el control.

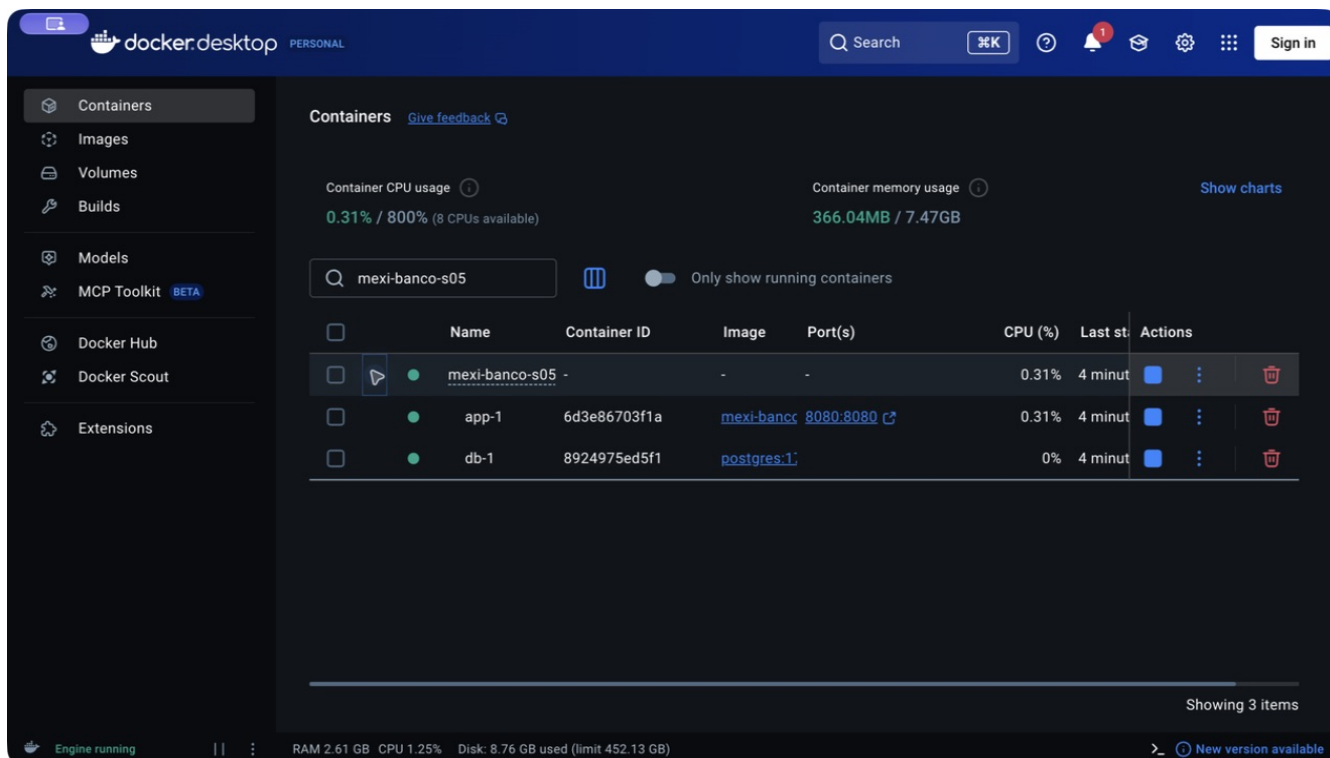


Figura 1. Captura real de Docker Desktop: el proyecto mexi-banco-s05 contiene app-1 y db-1; la aplicación publica el puerto 8080.

Comprobación	Resultado observado
Aplicación / base de datos	app: healthy · db: healthy
GET /actuador/health	HTTP 200; status: UP; grupos liveness y readiness
Entorno	macOS ARM64 · Docker 28.3.3 · Compose 2.39.2

La compilación terminó correctamente. El healthcheck verificó que la aplicación respondiera; que un contenedor esté en ejecución, por sí solo, no confirma que el servicio ya esté listo.

3. Pruebas de los endpoints

Se ejecutó la colección oficial **Mexi Banco v05.1**, en su orden original, con **Newman, el runner de Postman: 20 peticiones y 34 aserciones aprobadas, 0 fallos**. Las capturas de consultas corresponden al cliente de escritorio de Postman, sin iniciar sesión. La colección completa se ejecutó con Newman porque el cliente ligero exige una cuenta para gestionar colecciones.

El runner se conectó a `http://app:8080` dentro de la red Docker; Postman de escritorio usó `http://localhost:8080`. Ambos accedieron a la misma aplicación.

Prueba (peticiones agrupadas)	HTTP	Observación
Salud (1)	200	status UP.
Abrir origen y consultarlo (2)	201 / 200	Ana inicia con 1000.
Abrir destino (1)	201	Beto inicia con 500.
Transferir 200; consultar ambas cuentas y la transferencia (4)	201 / 200	Origen: 800; destino: 700; transferencia id 1.
SPEI de 50 y reintento con la misma clave (2)	201 / 201	Mismo id 1 y estado ENVIADO en ambas respuestas.
Consultar origen tras el SPEI (1)	200	Saldo 750: un solo cargo de 50.
Estado de cuenta y avisos (2)	200 / 200	3 movimientos; un solo SPEI_ENVIADO; 2 avisos.
Cuenta inexistente / CLABE repetida (2)	404 / 409	Errores de recurso inexistente y conflicto.
Alta sin CLABE / SPEI sin Idempotency-Key (2)	400 / 400	Peticiones incompletas rechazadas.
Saldo insuficiente / transferir a la misma cuenta (2)	422 / 422	Reglas de negocio rechazaron ambas operaciones.
Consultar saldo después de los errores (1)	200	Se mantiene en 750; no hubo cargos por errores.

Idempotencia: ambos envíos usaron prueba - 790407430998. Se devolvió la misma operación y los movimientos del origen sumaron $1000 - 200 - 50 = 750$. El destinatario interno conservó 700; el SPEI del ejemplo se dirige a una cuenta externa simulada.

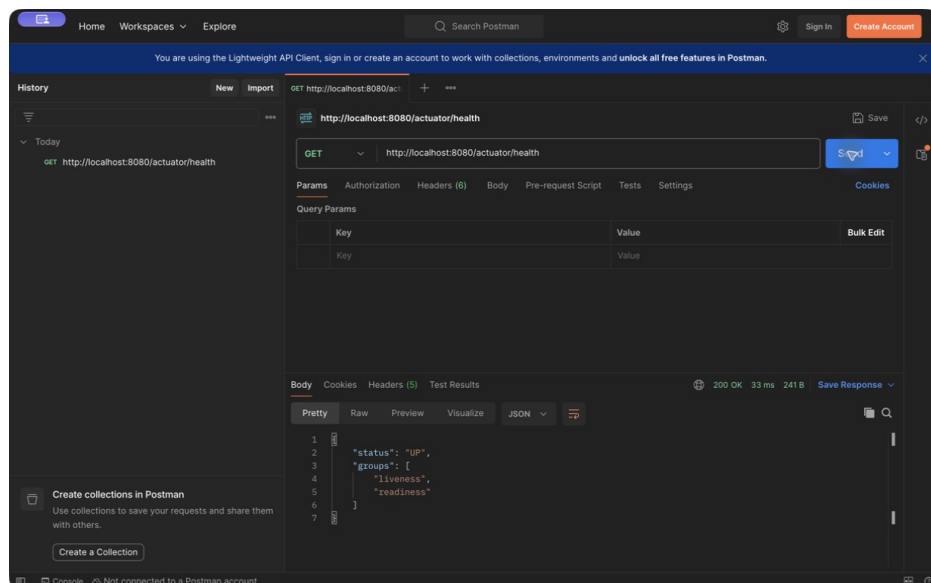


Figura 2. Consulta real de salud en Postman: HTTP 200 y status UP.

La CLABE de origen generada por la colección fue 002180790407430998; la de destino, 012180790407430998. Los cuerpos completos y las 34 verificaciones están en evidencia/resultado-postman.json.

4. Apagar con y sin volumen

Se volvió a consultar **la misma cuenta de origen** después de cada reinicio. Antes del experimento tenía saldo 750. En cada arranque se esperó a que app y db volvieran a estar saludables.

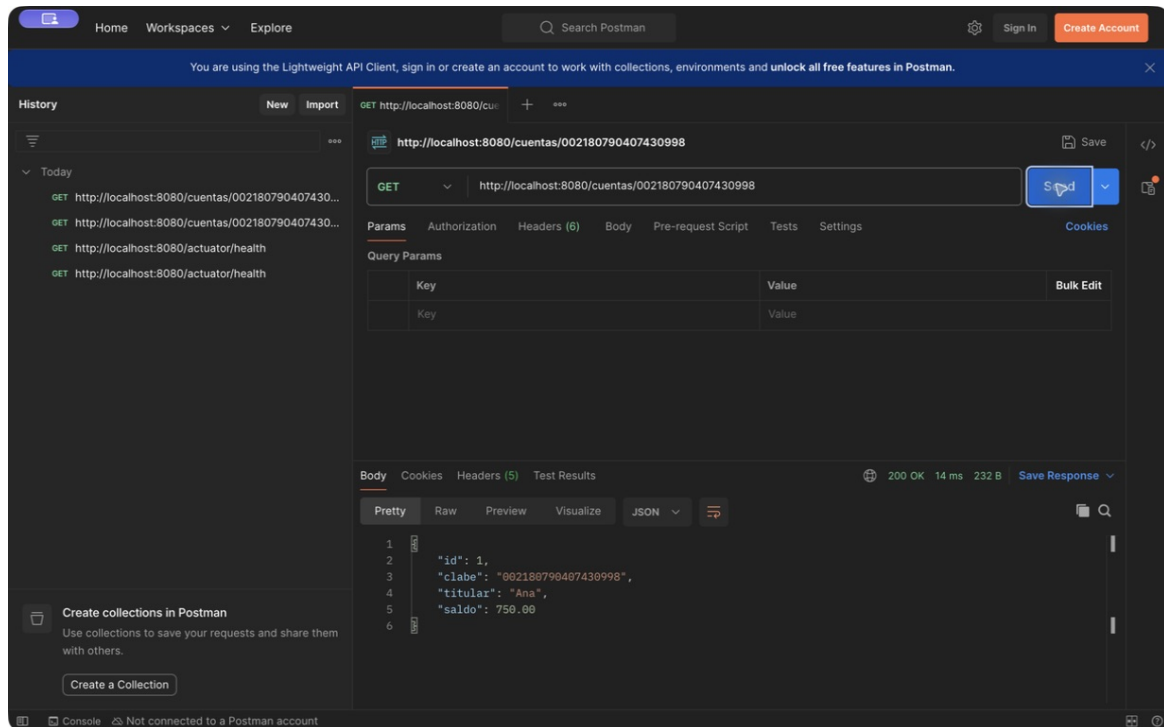


Figura 3. Después de docker compose down y up -d --wait: HTTP 200, misma cuenta y saldo 750.

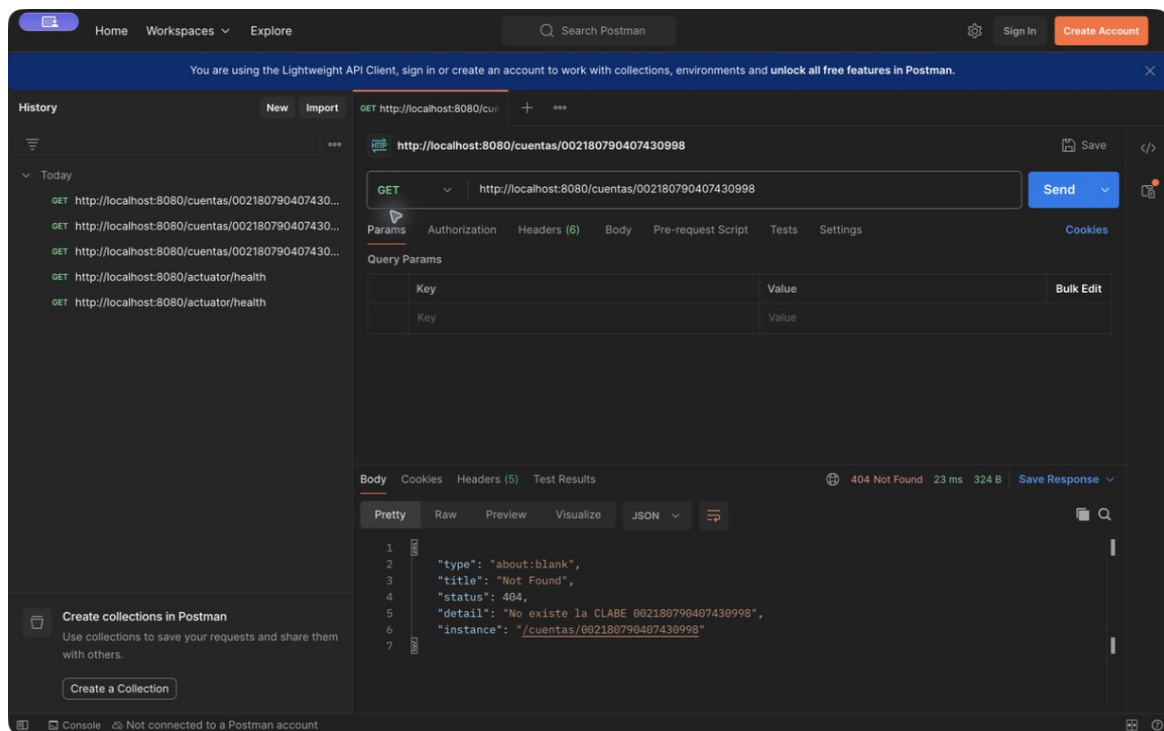


Figura 4. Después de docker compose down -v y up -d --wait: HTTP 404; la misma CLABE ya no existe.

Sin -v: se eliminaron los contenedores y la red, pero permaneció el volumen nombrado mexi-banco-s05_mexibanco_datos. PostgreSQL volvió a montar sus datos y la cuenta sobrevivió.

Con -v: también se eliminó ese volumen. El siguiente arranque creó uno nuevo; Hibernate reconstruyó las tablas, pero no las filas anteriores. Por eso la aplicación estaba saludable y aun así la cuenta devolvía 404.

5. Revisión del código

En `src/main/java/mx/mexibanco` se identificaron cinco módulos. Cada uno separa **controlador** (HTTP y validación), **servicio** (reglas y transacciones), **repositorio** (persistencia) y **entidad** (datos de una fila). Por ejemplo: `CuentaController`, `CuentaService`, `CuentaRepository` y `Cuenta`.

Módulo	Responsabilidad observada en el código
cuenta	Abrir y consultar cuentas; cargar y abonar saldos.
movimiento	Registrar asientos y consultar el historial; no expone edición ni borrado.
transferencia	Coordinar cargo, abono, movimientos y avisos en una transacción local.
notificacion	Guardar el aviso en su tabla y escribirlo en el log.
spei	Reservar la clave de idempotencia y procesar una transferencia externa simulada.

Fronteras: `TransferenciaService` llama a `CuentaService`, `MovimientoService` y `NotificacionService`; no accede a sus repositorios. Su método `@Transactional` confirma los cambios juntos o los revierte. Sigue siendo un **monolito modular**: un proceso de aplicación y una base de datos compartida.

SPEI: `SpeiTransacciones` reserva con `saveAndFlush` y una restricción `UNIQUE` antes del cargo. Si la clave ya terminó, `SpeiService` devuelve la solicitud existente. La latencia del otro banco se simula con una espera de 400 ms; no se realiza una transferencia bancaria real.

6. Compose y Dockerfile

Configuración	Qué hace
db / volumen	<code>postgres:17-alpine</code> ; <code>POSTGRES_DB</code> , <code>USER</code> y <code>PASSWORD</code> configuran el entorno docente. <code>mexibanco_datos</code> se monta en <code>/var/lib/postgresql/data</code> .
Salud y arranque	db: <code>pg_isready</code> cada 5 s. app espera <code>service_healthy</code> ; luego consulta <code>/actuador/health</code> cada 10 s con <code>wget</code> . Ambos: <code>timeout 3 s</code> , <code>retries 5</code> .
app / red	build: . usa el <code>Dockerfile</code> ; <code>8080:8080</code> publica HTTP. <code>DB_HOST=db</code> resuelve por DNS en la red <code>mexibanco</code> ; <code>DB_PORT=5432</code> . La base no publica un puerto al host.
Etape builder	JDK 21 Alpine; <code>WORKDIR /app</code> ; COPY de <code>pom.xml</code> , <code>mvnw</code> y <code>.mvn</code> ; <code>dependency:go-offline</code> descarga dependencias; COPY <code>src</code> ; <code>clean package -DskipTests</code> genera el JAR.
Etape de ejecución	JRE 21 Alpine; <code>WORKDIR /app</code> ; COPY <code>--from=builder</code> trae solo el JAR. EXPOSE 8080 documenta el puerto; <code>ENTRYPOINT</code> ejecuta <code>java -jar app.jar</code> .

Separar construcción y ejecución evita llevar el compilador y las herramientas de Maven a la imagen final. Copiar primero la configuración de Maven permite reutilizar la capa de dependencias. `-DskipTests` omite las pruebas unitarias durante el build; las 34 comprobaciones reportadas corresponden a la colección HTTP.

Conclusión. Compose permitió reconstruir el sistema de forma repetible; el volumen determinó la persistencia. La colección mostró saldos consistentes e idempotencia en los reintentos probados. Al separar módulos en procesos distintos, ya no bastará esta transacción local para coordinar todo.

Referencias: [Guía oficial S05 y colección de Postman](#) · [Código Mexi Banco v05.1](#). Evidencia obtenida el 26/09/2026.